Linguo-Economic Paradigms for Agentic Coding: A Neoclassical Framework for Autonomous Software Ecosystems

Part I: The Linguistic Architecture of Code: A Grammar for Agentic Composition

The foundational premise of this investigation is that the process of software creation, particularly when undertaken by autonomous AI agents, can be rigorously modeled and enhanced by drawing upon the structural and generative principles of human language. Specifically, this section establishes a novel framework wherein the creation of complex software functions is analogous to the linguistic process of neoclassical compounding. This analogy is not merely metaphorical; it provides a formal, computationally tractable grammar for generating modular, verifiable, and semantically rich code. By treating software primitives as linguistic morphemes and architectural rules as morphological processes, we can construct a system that guides AI agents in the composition of robust and elegant software artifacts. This part of the report will deconstruct this analogy, formalize it using established techniques from computational linguistics, and extend it to address the inherent heterogeneity of modern software ecosystems.

Section 1. Functions as Neoclassical Compounds: A New Lexicon for Software

This section lays the groundwork for the entire framework by deconstructing the principles of neoclassical compounding from linguistics and mapping them directly onto core concepts in software engineering. This mapping establishes a new lexicon for describing agent-generated code, transforming the act of programming from mere instruction-writing into a formal act of word-formation.

1.1. The Principles of Neoclassical Compounding

Neoclassical compounds represent a fascinating and highly productive category of word formation, particularly prevalent within scientific and technical domains.[1] Their structure and generative rules offer a compelling model for modular composition. The core principles of this linguistic process are as follows:

- **Classical Origins and Bound Morphemes:** Neoclassical compounds are formed from lexical elements—morphemes—borrowed from classical languages, primarily Ancient Greek and Latin.[1] A crucial characteristic of these elements is that they are typically "bound," meaning they cannot stand alone as independent words in the modern language.[1] For example,
  geo- and -logy are meaningful components but not free words in English; their full semantic potential is only realized through combination in a word like *geology*. This bound nature is a defining feature that necessitates a compositional grammar.
- **Combining Forms and Structural Roles:** Morphological theory provides a rich, albeit debated, terminology for these elements, including "confixes," "bound stems," and "formatives".[1] A useful classification by Bauer distinguishes between
  Initial Combining Forms (ICFs) and **Final Combining Forms (FCFs)**.[1] ICFs, such as bio- or hydro-, typically appear at the beginning of a compound and carry a high density of lexical information.[1] FCFs, such as
  -graphy or -metry, appear at the end and often define the semantic category or field of the new term.[5] Some forms can flexibly occupy either position, such as
  phobia in *arachnophobia* versus phobe in *phobophobe*.[1]
- **Linking Vowels and Productivity:** The composition of these forms is often mediated by a linking vowel, most commonly -o- (from Greek) or -i- (from Latin), which serves a structural or phonological purpose rather than a semantic one.[3] These vowels act as an interface, ensuring the two combining forms can be joined smoothly. The entire pattern of neoclassical compounding is remarkably productive, constantly generating new terms to name new concepts in science and technology, such as
  nutrigenomics or *ferrohydrodynamics*.[1] This productivity persists even among speakers who lack formal training in classical languages, which strongly suggests the existence of an internalized, rule-based system that can be abstracted and formalized.[1]

1.2. Mapping Linguistic Primitives to Software Primitives

The structural elegance and rule-based productivity of neoclassical compounding provide a powerful template for a new model of software composition. We propose a direct mapping from these linguistic concepts to the fundamental components of software engineering.

- **Initial Combining Forms (ICFs) as "Classical Primitives":** In our framework, ICFs correspond to **"Classical Primitives"**. These are the core, immutable, and highly optimized building blocks of software: fundamental algorithms (e.g., quicksort, dijkstra_pathfinding), core data structure operations (hash_lookup, tree_traversal), or low-level system calls (read_file, socket_connect). Like their linguistic counterparts, these primitives possess high "lexical density"—they encapsulate a significant amount of complex logic in a single unit.[1] However, they are often not standalone applications and require a broader context to be useful.
- **Final Combining Forms (FCFs) as "Modern Utilities":** FCFs map to **"Modern Utilities"**. These are higher-level, context-providing components, often specific to a particular framework, application domain, or output format. Examples include API wrappers (_to_json_response), user interface components (_render_react_component), database interaction layers (_save_to_postgres), or asynchronous task handlers (_dispatch_async_job). The Modern Utility gives the Classical Primitive its purpose and connects it to the larger application ecosystem.
- **Linking Vowels as "Adapters" or "Interfaces":** The linking vowel, the structural glue of neoclassical compounds, finds its direct analogue in the **"Adapter"** or **"Interface"** layer of software. This includes any "glue code" necessary to make a Primitive and a Utility interoperable. Examples are abundant: serialization/deserialization logic (e.g., a function that converts raw binary data from a Primitive into a JSON object for a Utility), data transformation layers (e.g., converting a numerical array into a pandas DataFrame), or any implementation of an abstract interface that allows two components to communicate without knowing the details of their respective implementations.

This mapping allows us to re-conceptualize the creation of hybrid software. A "hybrid word" in linguistics, which might combine a classical root with a native English word (e.g., *television*, from Greek tele- and Latin visio), is analogous to a **"hybrid function"** in our framework.[6] Such a function might combine a low-level Classical Primitive written in C++ for maximum performance with a high-level Modern Utility written in Python for ease of integration with a data science framework.

1.3. The "Compounded Function": A New Unit of Modularity

Building on this mapping, we define the central artifact of our framework: the **"compounded function."** This is a new, high-level software entity generated by an AI agent through the rule-based combination of one or more Classical Primitives and Modern Utilities, connected

by the necessary Adapters.

Consider the linguistic formation of *geology* from geo- (ICF), -o- (linking vowel), and -logy (FCF). The software equivalent follows the same pattern:

[primitive:timeseries_analysis] + `` + [utility:graphing_library_wrapper]

generate_trend_plot()

This compositional approach directly mirrors the **Composite design pattern** from software engineering.[8] In this pattern, a tree-like structure is built where both individual objects ("leaves") and compositions of objects ("composites") can be treated uniformly through a common interface.[8] In our model, Classical Primitives and Modern Utilities are the "leaf" nodes. The compounded function is the "composite" node, which provides a unified interface to the client, hiding the complexity of the underlying composition. The client can invoke

generate_trend_plot() without needing to know about the specific analysis algorithm or graphing library being used internally.

The true power of this model lies in its explicit enforcement of composition. The linguistic principle of "boundness"—the fact that combining forms like geo- and -logy cannot stand alone as words—is not a limitation but a powerful feature that drives the need for a compositional grammar.[1] Translating this to software provides a robust architectural constraint. By defining Classical Primitives as "bound" components, the framework can enforce a rule that they cannot be invoked directly from high-level application logic. They

must be wrapped within a compounded function. This forces the agent to create a safe, abstract, and contextually appropriate interface for every low-level operation. For instance, a raw cryptographic primitive, if defined as "bound," could not be called directly. An agent would be required to compound it with a utility that manages key rotation, handles exceptions, and logs usage, thereby creating a secure, high-level encrypt_data() function. The linguistic property of boundness thus becomes a formal mechanism for enforcing encapsulation and reducing the potential for misuse of low-level components, leading to a more robust and secure system architecture.

The following table provides a clear, one-to-one reference for the core analogy that underpins the entire report, making the abstract concept concrete and serving as a foundational lexicon for the remainder of the analysis.

Table 1: Mapping Linguistic Compounding to Software Composition

Linguistic Concept	Description	Software Analogue	Description & Example
Neoclassical Compound	A new word formed from classical elements (e.g., *biography*).	**Compounded Function**	A new, high-level function created by an agent. (visualize_data_from_api)
Initial Combining Form (ICF)	Bound morpheme, high lexical density (e.g., bio-, geo-).	**Classical Primitive**	Core, reusable algorithm or data operation. (fetch(), sort(), encrypt())
Final Combining Form (FCF)	Bound morpheme, provides context/category (e.g., -logy, -graphy).	**Modern Utility**	Framework-specific output or action. (_to_json(), _render_view(), _save_to_db())
Linking Vowel	Structural connector, no semantic meaning (e.g., -o-).	**Adapter / Interface**	Glue code, data-mapper, serialization layer. (json.loads(), DataFrame())
Productivity	The pattern is actively used to create new technical terms.	**Agentic Generation**	The framework enables agents to autonomously generate novel, useful functions.

Section 2. A Formal Grammar for Code Compounding: Finite-State

Morphology

To move from a conceptual analogy to an operational framework, we must define a formal grammar that AI agents can use to perform code compounding. The field of computational linguistics provides a powerful and well-suited tool for this purpose: the Finite-State Transducer (FST). This section proposes an FST-based architecture that operationalizes the linguistic model, enabling the efficient, rule-based, and formally verifiable generation of compounded functions.

2.1. Finite-State Transducers in Computational Morphology

Finite-State Transducers are a generalization of finite-state automata that define relationships between two sets of strings.[10] While an automaton accepts or rejects a single string (defining a regular language), a transducer reads an input string from one "tape" and writes a corresponding output string to another "tape," thereby defining a regular relation.[12] This capability makes them exceptionally well-suited for morphological analysis and generation.[13]

The key properties of FSTs that make them ideal for our framework are:

- **Bidirectionality:** A single FST can be run in both directions. In morphology, this means the same transducer can be used for analysis (mapping a surface word like "cats" to its lexical form cat+N+PL) and for generation (mapping cat+N+PL to "cats").[15]
- **Efficiency:** FST operations are computationally efficient, making them suitable for real-time applications. Implementations are highly optimized for speed and memory usage.[13]
- **Compositionality:** FSTs are closed under the operation of composition. This means that a sequence of transducers, where the output of one becomes the input of the next, can be compiled into a single, equivalent transducer that performs the entire cascade of operations in one pass.[12] This property is fundamental to building complex systems from simple, verifiable rules.

2.2. An FST-based Architecture for Agentic Coding

We propose a generative architecture for agentic coding that leverages the compositionality of FSTs. The system is built upon two primary types of transducers: a Lexicon FST and a set of

Rule FSTs.

1. **The Lexicon FST:** This transducer serves as the foundational dictionary of the agent's coding language. It defines the mapping between abstract semantic concepts and their concrete implementations as Classical Primitives.
   - **Input Tape (Lexical/Semantic Level):** The input side of the Lexicon FST accepts a sequence of abstract "morphemes" that describe a desired function. For example, sort^list^descending.
   - Output Tape (Intermediate Code Level): The output side maps this abstract representation to an "intermediate form" of the code. This intermediate form consists of the concrete function call for the corresponding Classical Primitive, along with morpheme boundary markers. For example, my_lib.quick_sort(direction=DESC).
     The Lexicon FST thus contains the entire library of available, verified primitives and acts as the bridge between an agent's high-level intent and low-level code.
2. **Rule FSTs:** These transducers encode the architectural and non-functional requirements of the system. Each Rule FST implements a specific cross-cutting concern, such as error handling, logging, security, or data validation. They operate on the intermediate code produced by the Lexicon FST.
   - **Analogy to Morphological Rules:** This is directly analogous to the use of rule FSTs in linguistics to handle spelling changes (orthographic rules).[12] For example, a linguistic rule might state "insert an
     e after a stem ending in x before the plural morpheme s" (fox^s

     foxes).[18]
   - **Software Rule Example:** A software-based Rule FST could implement an architectural pattern like "all external API calls must be wrapped in a try/except block and log failures to a specific service." This transducer would take an intermediate form like api.fetch_data() as input and produce the compounded output:
     Python
     ```
     try:
       result = api.fetch_data()
     except APIError as e:
       logger.error(f"API call failed: {e}")
       result = None
     ```

Other Rule FSTs could enforce security policies (e.g., sanitizing all inputs), add performance monitoring hooks, or handle data format transformations.

2.3. The Composition Process

The complete generation of a compounded function is achieved through the composition of these FSTs. The process unfolds as follows:

1. **Specification:** An AI agent begins with a high-level goal, such as "securely fetch user data from the remote API and display it in the user interface."
2. **Lexicalization:** This goal is translated into a sequence of abstract morphemes that the Lexicon FST understands: fetch_user_data^auth^secure^json_parse^display_table.
3. Composition and Transduction: This lexical string is processed through a single, pre-compiled transducer that is the composition of the Lexicon FST and all relevant Rule FSTs, applied in a specific order: $T_{final} = T_{Lexicon} \circ T_{SecurityWrapper} \circ T_{ErrorHandling} \circ T_{Logging} \circ T_{UIAdapter}$
4. **Final Output:** The output of this composed transducer, , is the final, well-formed, and architecturally compliant compounded function, ready for integration and execution.

This FST-based approach transforms code generation from a monolithic, probabilistic task into a deterministic and structured process of linguistic composition. A significant advantage of this formalization is the inherent verifiability it provides. Standard code generation by Large Language Models (LLMs) is powerful but lacks formal guarantees; it can produce code that appears correct but contains subtle flaws or fails to adhere to architectural constraints.[19] Our FST-based framework, in contrast, aligns with principles from formal methods and compositional reasoning.[21]

Each individual FST, whether from the lexicon or the rule set, is a formal mathematical object. It is possible to prove specific properties about each transducer. For example, one can

formally verify that T_{ErrorHandling} guarantees that its output code will never raise an unhandled exception of a certain type. This is a form of assume-guarantee reasoning: the error-handling transducer *assumes* it receives a valid function call and *guarantees* it will be wrapped in an exception block.[23] Because the composition of FSTs preserves these properties, the final composed transducer,

, inherits the guarantees of its constituent parts. The very act of generating code becomes a constructive proof of that code's adherence to a set of pre-verified architectural properties. This elevates agentic coding from a purely generative discipline to one that is grounded in the principles of high-assurance software engineering, providing a much higher degree of confidence in the correctness and robustness of the resulting system.

Section 3. Cross-Lingual Alignment for a Universal Function Space

The FST-based grammar provides a powerful mechanism for composing functions within a single, well-defined ecosystem. However, real-world software development is characterized by profound heterogeneity. Developers and agents must navigate a landscape of different programming languages (Python, Rust, Java), different libraries that perform the same task (requests vs. httpx), different versions of the same API, and different idiomatic coding styles. To create a truly collaborative and effective multi-agent system, we must equip agents with the ability to understand and reason about functionality across these boundaries. This section proposes leveraging techniques from multilingual Natural Language Processing (NLP), specifically cross-lingual alignment, to create a universal semantic space for software functions.

3.1. The "Many Languages" Problem in Software

The challenge of heterogeneity in software mirrors the challenge of multilinguality in human language. For an AI agent to effectively reuse existing code or collaborate with other agents, it needs a way to determine functional equivalence despite superficial differences in implementation. It must understand that a Python function using the requests library and a JavaScript function using fetch can both satisfy the semantic goal of "making an HTTP GET request." They are, in essence, "translations" of the same underlying concept. Without this understanding, agents would be confined to their own narrow silos, unable to leverage the vast ecosystem of existing software or collaborate with agents using different toolchains.

3.2. Cross-Lingual Alignment in Multilingual Models

In NLP, cross-lingual alignment is the process of learning a shared representation space (an embedding space) where words, phrases, or sentences with similar meanings in different languages are mapped to nearby vectors.[24] This alignment is the critical technology that enables zero-shot cross-lingual transfer, a paradigm where a model trained to perform a task (e.g., sentiment analysis) in a high-resource language like English can successfully perform the same task in a low-resource language without any direct training on that language.[24]

This alignment is typically achieved through techniques like contrastive learning. During training, the model is presented with pairs of translated sentences. A contrastive loss function then explicitly encourages the model to pull the vector representations of these translation pairs closer together in the embedding space, while simultaneously pushing them further away from the representations of non-translated sentences.[27] The result is a geometrically organized semantic space where distance corresponds to meaning similarity, irrespective of the source language.

3.3. Building a Semantic Space for Functions

We propose adapting this powerful technique to the domain of software engineering by treating individual functions as the fundamental unit of "translation." The goal is to create a **universal function space** where functions from any language, library, or framework can be represented and compared based on their semantic meaning.

- **Representing Function "Meaning":** The semantic content of a function is multi-faceted and can be extracted from several sources. A rich vector representation for a function can be created by encoding and combining information from its:

    - Name and docstring (natural language content).
    - Parameter names, types, and default values.
    - Return type.
    - A graph-based representation of its Abstract Syntax Tree (AST), capturing its control flow and logic.
    - The dependency context in which it is used.
- **Training for Alignment:** A large-scale language model can be trained on a massive dataset of "translation pairs" of functions. These pairs would consist of functions from different libraries or languages that are known to perform the same task. Such a dataset could be curated from sources like Rosetta Code, open-source projects with parallel implementations, or by identifying functions that are frequently used to solve the same problem on platforms like Stack Overflow. Using a contrastive learning objective, the model would learn to map, for example, a Python csv.reader function and a Java CSVParser function to very close points in the high-dimensional vector space.
- **A Semantic Search Engine for Code:** The resulting universal function space acts as a powerful semantic search engine. An AI agent can formulate a query in natural language (e.g., "a function to compute the SHA256 hash of a string") or provide an example implementation. The system would then perform a nearest-neighbor search in the embedding space to retrieve a list of semantically similar functions from across the entire ecosystem. These results can be ranked not only by semantic similarity but also by the economic and risk metrics that will be developed in Part II of this report, allowing the agent to make a sophisticated, multi-objective choice.

The distinction between "weak" and "strong" alignment, a concept from the NLP literature, has profound implications for the reliability of such a system.[25]

Weak alignment requires only that the nearest neighbor of a function in a target language is its correct translation. For example, the vector for requests.get() should be closer to httpx.get() than it is to any other JavaScript HTTP client function. This is sufficient for basic retrieval.

However, for building functionally correct and reliable software, **strong alignment** is necessary. Strong alignment imposes a much stricter geometric constraint: the representation of a function must be closer to its translations than it is to *any other function*, including those within its own language or library.[25] This means the vector for

requests.get() must be closer to httpx.get() than it is to requests.post(). Many subtle software bugs arise from using a function that is semantically close but functionally incorrect for a given task—for instance, using a path-joining function intended for file systems when one for URLs is required. These functions are conceptually similar ("joining strings to form a path") but have different rules for handling slashes and protocols, leading to incorrect behavior.

By explicitly training the model to achieve strong alignment, the system is forced to learn these fine-grained semantic distinctions that are critical for correctness. The contrastive

learning process must push the representations of requests.get() and requests.post() apart, encoding the crucial difference between idempotent retrieval and state-changing submission. This makes the semantic search for functions far more robust. The agent's query for a "data retrieval function" is less likely to be contaminated by semantically adjacent but functionally incorrect alternatives. Achieving strong alignment in the universal function space is therefore a critical step in moving from simple code retrieval to reliable, agent-driven software composition.

Part II: The Economic Dynamics of a Multi-Agent Coding Ecosystem

Having established a linguistic framework for the composition of code, this report now shifts its analytical lens. We move from viewing code as a grammatical structure to viewing it as a collection of assets within a dynamic economy. In this paradigm, AI agents are not merely rule-following composers but are modeled as rational (or, more realistically, boundedly rational) economic actors. Their individual and collective behavior—the decisions to create, reuse, refactor, or depend on a piece of code—can be analyzed and optimized using the principles of neoclassical economics, finance, and game theory. This part of the report will develop this economic model, demonstrating how concepts like marginal utility, risk-return tradeoffs, and strategic interaction can be used to guide a multi-agent system toward the creation of efficient, robust, and valuable software.

Section 4. The Function as an Economic Asset: Utility and Value

At the heart of microeconomics is the study of how individuals make decisions under scarcity. This section applies the foundational theory of marginal utility to model the decision-making process of an individual AI coding agent. By quantifying the "utility" of a function and the "cost" of its creation, we can establish a rational economic basis for an agent's actions, from feature development to code optimization and the management of technical debt.

4.1. The Theory of Marginal Utility

Marginal utility is a cornerstone of neoclassical economics, describing the additional

satisfaction or benefit (utility) a consumer derives from consuming one more unit of a good or service.[30] The theory is built upon a key observation about human behavior known as the

Law of Diminishing Marginal Utility. This law states that as consumption of a good increases, the marginal utility gained from each additional unit tends to decrease.[32] For example, the first slice of pizza consumed by a hungry individual provides a great deal of utility; the fifth slice, however, provides significantly less.

Economists assume that rational consumers aim to maximize their total utility given a limited budget. They will continue to consume a good as long as the marginal utility gained is greater than or equal to the marginal cost of acquiring it.[31] The optimal allocation of resources is achieved when the last dollar spent on every good yields the same amount of marginal utility. This principle provides a powerful framework for understanding and predicting choice.

4.2. An Agent's Utility Function for Code

We can directly apply this framework by modeling an AI coding agent as a producer and consumer of software functions. The agent operates within an economy of limited resources, its "budget" consisting of:

- **Computation Time:** The CPU/GPU cycles required for analysis, generation, and testing.
- **LLM Tokens:** The finite number of tokens that can be processed in prompts for code generation or analysis.
- **Complexity Budget:** A constraint on the overall complexity (e.g., cyclomatic complexity, dependency count) that can be added to the system, reflecting the long-term maintenance burden.

The "utility" that an agent derives from a function is not a single value but a multi-attribute vector representing its quality and value to the system. A function's total utility,

, can be expressed as a weighted sum of several key attributes:

When an agent considers an action—such as creating a new function, refactoring an existing one, or adding a test case—it performs a cost-benefit analysis by evaluating the **marginal utility** of that action. This is calculated as the change in total system utility divided by the resources expended:

An agent, acting as a rational economic entity, will prioritize tasks that offer the highest marginal utility per unit of resource cost and will cease working on a task when its marginal utility falls below its marginal cost.

4.3. Diminishing Returns in Software Development

The Law of Diminishing Marginal Utility is readily observable in software engineering. Consider the task of optimizing a function's performance. The initial effort that reduces its execution time from 1 second to 100 milliseconds yields an enormous marginal utility, as it may transform the user experience from unacceptable to interactive. A subsequent optimization that takes the function from 100ms to 50ms still provides significant, though lesser, utility. However, an exhaustive effort to reduce the time from 10 microseconds to 5 microseconds, while technically an improvement, may have a marginal utility approaching zero for a typical

web application, yet it could consume substantial developer and computational resources.[33]

This economic model provides a rational basis for agents to make crucial development decisions. It allows them to autonomously determine when a feature is "good enough," when an optimization is no longer cost-effective, and when the pursuit of perfection yields diminishing returns. By constantly evaluating MU against MarginalCost, the agent collective can avoid common pitfalls like premature optimization, feature creep, and gold-plating, leading to a more efficient allocation of resources across the entire project.

This framework offers a particularly powerful and formal model for understanding and managing **technical debt**. In software engineering, technical debt is the implied cost of rework caused by choosing an easy (limited) solution now instead of using a better approach that would take longer. The concept of **negative marginal utility**, where consuming an additional unit actually decreases total satisfaction, provides a precise economic definition for this phenomenon.[30]

An action that introduces technical debt is one with a negative long-term expected marginal utility. For example, an agent might be able to quickly implement a new feature by copy-pasting a large block of code, adding a function with no documentation or tests, or using a deprecated library. The immediate, short-term utility of this action is positive—the feature is delivered. However, a sophisticated utility model can project the future costs associated with this decision. The duplicated code will increase future maintenance costs, as bugs will need to be fixed in multiple places. The untested function increases the probability of a production failure. The deprecated library introduces a security risk and will necessitate a costly migration in the future.

When these future negative utilities are discounted to their present value and summed, the total expected marginal utility of the "hacky" solution becomes negative (MU < 0). A rational agent programmed to maximize long-term utility would therefore reject this implementation, not because it violates an abstract principle of "clean code," but because it is an economically unsound decision. This transforms the often-vague goal of managing technical debt into an explicit optimization problem, allowing agents to make quantifiable, defensible tradeoffs between short-term velocity and long-term system health.

Section 5. Risk, Return, and the Code Portfolio: A CAPM for Software Dependencies

While marginal utility theory provides a model for an agent's internal decision-making, it does not fully capture the risks inherent in a highly interconnected software ecosystem. Modern software is rarely built in isolation; it is assembled from a vast network of open-source

dependencies.[34] This section introduces a novel approach to managing this external risk by modeling the software ecosystem as a financial market and applying the Capital Asset Pricing Model (CAPM) to evaluate the risk-return profile of software components.

5.1. Systemic Risk in Software Ecosystems

The open-source software ecosystem is a complex adaptive system built on dependency networks of immense scale. Analysis of repositories like Maven Central (for Java) and npm (for JavaScript) reveals that these networks are not random. They consistently exhibit topologies characteristic of other complex networks, such as social networks or the internet itself, namely **scale-free** and **small-world** properties.[36]

A scale-free network is characterized by a power-law degree distribution, meaning it has a small number of highly connected nodes, or "hubs," and a large number of nodes with very few connections.[38] In software, these hubs are foundational libraries and frameworks—like

log4j, junit, or react—that are dependencies for millions of other projects.[36] While this topology promotes efficient code reuse, it also creates profound

systemic risk. A single failure—such as a critical security vulnerability or a breaking change—in one of these central hubs can trigger **cascading failures** that propagate throughout the entire ecosystem, affecting a vast number of downstream applications.[36] The 2021 Log4Shell vulnerability in the

log4j library is a canonical example of this phenomenon, forcing a global, emergency patching effort across the industry.[36]

5.2. The Capital Asset Pricing Model (CAPM)

The Capital Asset Pricing Model (CAPM) is a foundational model in finance used to determine the theoretically appropriate required rate of return for an asset, given its level of risk.[45] The model's central insight is to differentiate between two types of risk:

- **Systematic Risk:** This is market-wide, non-diversifiable risk that affects all assets in the market. It stems from macroeconomic factors like interest rate changes or recessions. Investors are compensated for bearing this risk because it cannot be eliminated through portfolio diversification.
- **Unsystematic Risk:** This is asset-specific risk that is unique to an individual company or

security (e.g., a product recall or a factory strike). This risk can be effectively eliminated by holding a well-diversified portfolio.

CAPM posits that the expected return of an asset is a function of the risk-free rate of return, the expected return of the market as a whole, and the asset's sensitivity to systematic risk.

This sensitivity is quantified by the variable **Beta (** **)**. An asset

with a of 1 moves in lockstep with the market. An asset with a

is more volatile than the market, offering potentially higher

returns but also greater risk. An asset with a is less volatile.[46]
The CAPM formula is:

where is the expected return of

the asset, is the risk-free rate, and

is the expected return of the market.

5.3. A CAPM for Code Dependencies

We propose a direct mapping of these financial concepts to the software dependency ecosystem to create a quantitative framework for risk management.

- **The Market:** The entire open-source ecosystem under consideration (e.g., Maven Central, PyPI). The "market return" ( ) represents the aggregate utility growth of the entire ecosystem.
- **An Asset:** An individual software component, library, or compounded function.
- **Return ( ):** The utility provided by the software asset, as

defined by our multi-attribute utility function (performance, features, correctness, etc.).

- **Systematic Risk:** The risk stemming from the function's dependencies on the central, "hub" packages of the ecosystem. A vulnerability in a core framework that a function depends on is a systematic shock. This risk cannot be "diversified away" by the agent if that dependency is essential for its functionality.
- **Unsystematic Risk:** The risk of bugs or flaws within the function's own implementation, independent of its dependencies. This risk is "diversifiable" in the sense that an agent can mitigate it through its own actions, such as improved testing, formal verification, or better implementation.

- **Beta ( ) of a Function:** A function's beta is a calculated measure of its sensitivity to systemic risk. It is a function of its position in the dependency graph. A function that has a deep and complex transitive dependency chain relying on multiple high-centrality hub packages will have a high beta. Conversely, a self-contained function with few or no external dependencies will have a beta close to 0.

5.4. Agent as Portfolio Manager

This CAPM framework transforms the AI agent's role from a simple code composer to a sophisticated **portfolio manager**. When an agent needs to select a library to fulfill a specific functional requirement, it is faced with a choice analogous to an investor selecting a stock. It can query the universal function space and receive multiple candidates that are semantically equivalent. Instead of choosing based on features alone (the "return"), the agent can now make a risk-adjusted decision.

For example, to implement an HTTP client, the agent might consider two libraries:

- **Library A:** A feature-rich, high-performance library that is part of a massive, popular web framework. It offers high "return" but also has a high **beta** because it transitively depends on dozens of other packages, including several major ecosystem hubs.
- **Library B:** A lightweight, standalone library with fewer features but zero dependencies. It offers a lower "return" but has a **beta** of 0.

The agent, guided by a system-wide risk tolerance parameter set by a human overseer, can now make a rational, economic choice. For a non-critical internal tool, it might choose Library A to maximize development speed. For a high-assurance, mission-critical security component, it would choose Library B to minimize exposure to systemic risk. The agent constructs the final application from a "portfolio" of dependencies that is optimized for a desired point on the risk-return frontier.

This framework allows for the quantification and "pricing" of the systemic risk that a newly published component *introduces* into the ecosystem. When an agent creates and publishes a new library, its beta can be calculated by analyzing its dependency graph. Furthermore, graph-based simulations, akin to financial stress tests, can be run to model how the failure of this new library would propagate through the ecosystem.[36] This analysis yields a "Systemic Risk Contribution" score. This score can be integrated into a reputation system or a resource allocation mechanism. Agents that consistently publish low-beta, resilient components that strengthen the ecosystem could be rewarded with higher priority or more computational resources. Conversely, agents that introduce high-risk, fragile components could be penalized. This creates a powerful, market-based incentive structure that encourages behavior beneficial to the long-term health and stability of the entire software ecosystem.

Section 6. Strategic Interaction and Workflow Optimization: A Game-Theoretic Approach

In a sophisticated software development environment, AI agents do not operate in a vacuum. They form a society of actors whose decisions are interdependent. They must collaborate on shared goals, compete for finite resources, and negotiate over interfaces and standards.[47] To understand and optimize these complex interactions, we turn to game theory, the mathematical study of strategic decision-making. This section models the multi-agent coding workflow as a series of "games," allowing us to design mechanisms that align the self-interest of individual agents with the global objectives of the system.

6.1. Multi-Agent Systems (MAS) as a Game

Game theory provides a formal framework for analyzing strategic interactions among rational agents.[49] The core elements of a game—players (agents), actions (strategies), and payoffs (utilities)—map directly onto our multi-agent coding ecosystem. An agent's "strategy" could be its choice of which bug to fix, which library to use, or how to implement a new feature. Its

"payoff" is the change in its utility function, which might be based on the successful integration of its code, the performance improvement it delivers, or a reputation score. Because the outcome for one agent depends on the strategies chosen by all other agents, their interactions can be modeled and analyzed as a game.

6.2. Modeling Cooperation and Competition

The interactions within the agent society can be a mix of cooperation and competition, which can be modeled using different types of games:

- **Competition (Zero-Sum or General-Sum Games):** Agents often compete for scarce resources. This could include priority access to a limited number of CI/CD runners, a finite computational budget for running tests, or the allocation of a human developer's limited time for code review. This competition, if unmanaged, can lead to globally suboptimal outcomes, such as agents engaging in "races" to push code quickly at the expense of quality, or hoarding useful private functions instead of sharing them.
- **Cooperation (Coordination Games):** The ultimate goal of building a functional, high-quality application is a shared one, requiring extensive cooperation. Agents must coordinate to agree on stable API contracts, collaborate on debugging complex cross-component issues, and collectively maintain the health of the codebase. This is a coordination game, where the highest payoffs are achieved when all agents adopt compatible strategies.

A key concept from game theory for analyzing such systems is the **Nash Equilibrium**. A system is in a Nash equilibrium if no single agent can improve its own payoff by unilaterally changing its strategy, given the strategies of all other agents.[48] While a system may have many possible Nash equilibria, not all of them are desirable. For example, a state where no agent submits code because the review process is too onerous could be a stable but unproductive equilibrium. The primary goal of applying game theory here is not just to analyze the existing dynamics, but to actively design the "rules of the game" to ensure that the most desirable, cooperative, and productive behaviors constitute a stable Nash equilibrium.

6.3. Mechanism Design for Optimal Collaboration

Mechanism design is a subfield of game theory that focuses on designing these "rules of the game" to incentivize individual agents to behave in a way that is beneficial for the system as a whole.[50] By carefully crafting the incentive structures, we can align the local, self-interested decision-making of each agent with the global system goals of efficiency, robustness, and

quality.

Several mechanisms can be applied to optimize the multi-agent coding workflow:

- **Resource Allocation via Auctions:** To manage contention for scarce resources like testing environments or deployment slots, the system can implement an auction mechanism. When an agent wants to merge a change, it submits a "bid" based on the predicted marginal utility of its change. A **Vickrey (second-price) auction**, where the winner pays the price of the second-highest bid, can be used to incentivize agents to bid their true valuation, leading to an efficient allocation where the most valuable changes are prioritized.[50]
- **Fair Credit Assignment via Shapley Value:** In collaborative efforts, fairly distributing credit is crucial for incentivizing cooperation. The **Shapley value** provides a principled method for this. It calculates each agent's contribution to a collective outcome by averaging its marginal contribution across all possible subgroups (coalitions) of agents.[50] An agent's reputation score or reward could be tied to its Shapley value, thus rewarding agents who create highly reusable and impactful components that benefit many different parts of the project. This discourages "free-riding" and encourages the creation of valuable public goods within the codebase.
- **Negotiation over Interfaces:** When two agents need to interact, they must agree on an API contract. This can be modeled as a negotiation game. The agents can use automated negotiation protocols to propose and counter-propose interface designs, searching for a **Pareto-optimal** outcome where the final contract cannot be changed to make one agent better off without making the other worse off.[50] This allows for the decentralized evolution of system interfaces without requiring a central architect.

The process of code review and integration provides a particularly rich domain for game-theoretic modeling. While traditional models assume perfect rationality, human and artificial agents operate with **bounded rationality**—their knowledge, computational capacity, and time are limited.[51] This suggests that a simple optimization model is insufficient. Instead, the social dynamic of code review can be more accurately modeled as a

Keynesian Beauty Contest.[51]

In this game, contestants are asked to pick the most beautiful face from a set of photographs, with the winner being the one whose choice is closest to the *average* choice of all contestants. The optimal strategy is therefore not to pick the face one personally finds most beautiful, but to predict which face the other contestants will find most beautiful. This leads to recursive, "level-k" reasoning: a level-0 thinker picks their favorite; a level-1 thinker picks what they think the average level-0 thinker will pick; a level-2 thinker models the behavior of level-1 thinkers, and so on.[51]

This maps directly to multi-agent code review. An agent submitting a pull request is trying to

get it approved by a group of other agents.

- A **level-0** agent writes code that it believes is correct based on its own internal logic.
- A **level-1** agent anticipates the automated checks of the CI system and writes code that passes the linter and unit tests.
- A **level-2** agent thinks, "What will the security-focused Agent C, who I know is a reviewer, look for?" It then preemptively scans its own code for potential vulnerabilities that Agent C is likely to flag.
- A **level-3** agent might reason, "Agent C is very strict, but Agent D (focused on performance) often approves changes quickly if they improve latency. I will highlight the performance gains in my description to appeal to Agent D, hoping their approval will influence Agent C."

This model implies that to be successful, agents must maintain dynamic mental models of their peers' priorities, biases, and decision criteria. An agent's strategy for writing "good" code is not just about objective correctness, but about producing an artifact that it predicts will be acceptable to the community of reviewers. This leads to the emergence of implicit coding conventions, shared standards, and a more sophisticated, socially aware coding process that mirrors the dynamics of human software development teams.

The following table acts as a concise summary of the core ideas in Part II, demonstrating how each economic model is not just an abstract analogy but a practical tool for solving a concrete problem in the multi-agent coding workflow.

Table 2: Economic Models for Agentic Coding Optimization

Economic Model	Core Principle	Problem in Agentic Coding	Proposed Solution
Marginal Utility Theory	Diminishing returns; maximize satisfaction within a budget.	Code bloat, over-optimization, defining "good enough."	Agents calculate the marginal utility of each code change vs. its resource cost. They stop when

			, preventing wasted effort and formalizing technical debt as negative utility.
Capital Asset Pricing Model (CAPM)	Risk-return tradeoff; pricing of systematic (market) risk.	Managing dependency risk; avoiding cascading failures.	The ecosystem is a "market." Functions have a "beta" based on their dependency on systemic hubs. Agents build a "portfolio" of dependencies to balance risk and return.
Game Theory & Mechanism Design	Strategic interaction; aligning individual incentives with global goals.	Inefficient resource allocation, lack of cooperation, code hoarding.	Design "games" (auctions for CI/CD time, reputation systems based on Shapley value) that make cooperation the dominant strategy for self-interested agents.

Part III: Synthesis and Emergent Paradigms

Having established the distinct linguistic and economic frameworks, this final part of the report integrates them into a single, cohesive paradigm. Here, we explore the novel software development lifecycle that emerges when functions are treated simultaneously as compounded words and as economic assets. This synthesis leads to a self-organizing, adaptive, and evolving software ecosystem. We will describe the core workflow of this system, analyze its emergent properties, and speculate on the long-term future of software engineering in a world where human developers transition from being artisans of code to being architects of economies.

Section 7. The Linguo-Economic Paradigm: A Unified Framework

The power of the proposed paradigm lies in the fusion of its two constituent parts. The linguistic framework provides a generative engine for creating structured, verifiable code, while the economic framework provides a rational evaluation and selection mechanism for managing that code. This creates a continuous, closed-loop system for software development.

7.1. The Generative-Evaluative Loop

The core workflow of the linguo-economic paradigm is a generative-evaluative loop that drives the evolution of the software system:

1. **Generation (The Linguistic Phase):** The process begins when a task is assigned to an agent or a group of agents. This task is a high-level description of a required functionality. The agent system first consults the **universal function space** to find existing Classical Primitives and Modern Utilities that are semantically relevant to the task. It then uses the formal grammar, implemented via **Finite-State Transducers (FSTs)**, to compose these components into one or more candidate "compounded functions." This generation process is not random; it is constrained by the architectural rules encoded in the Rule FSTs, ensuring that every candidate is, by construction, compliant with system-wide standards for error handling, security, and logging.
2. **Evaluation (The Economic Phase):** The set of generated candidate functions is then subjected to a rigorous economic evaluation. Each candidate is treated as a potential

"investment." The agent collective, or a specialized "evaluator" agent, calculates a comprehensive value score for each candidate based on the economic models developed in Part II:

- **Utility Calculation:** The multi-attribute utility of the function (performance, readability, etc.) is estimated.
- **Risk Assessment:** The function's **Beta ( )** is calculated by analyzing its proposed dependency graph to quantify its exposure to systemic risk.
- **Cost Analysis:** The resources required for its ongoing maintenance and execution are projected.

3. **Selection and Integration:** The candidate function that offers the highest risk-adjusted utility is selected for integration into the main codebase. This is a portfolio management decision, balancing the immediate functional return against the long-term risk and cost. Once verified and integrated, the new compounded function is itself added to the universal function space, becoming a new, higher-level primitive that can be used by other agents in future compositions.

This loop—generate via linguistic rules, evaluate via economic principles, select via rational choice—is the engine of development in the linguo-economic paradigm. It is a continuous, autonomous process of proposing, vetting, and integrating code that drives the system's evolution.

7.2. Emergent Properties of the System

When this generative-evaluative loop is run at scale with a multitude of interacting agents, several powerful, high-level properties are expected to emerge, shaping the entire ecosystem in a bottom-up fashion.

- **A Living Lexicon:** The set of available functions in the universal function space is not a static library. It behaves like the lexicon of a living language. As agents create, test, and deploy new compounded functions, the most successful ones—those that are frequently reused and demonstrate high utility and low risk—become entrenched "words" in the

system's vocabulary.[4] They are the common currency of development. Conversely, functions that are superseded by better alternatives, prove to be risky (high-beta), or simply fall out of use will become "archaic." The system can track the usage and utility of all functions, eventually "deprecating" and "garbage-collecting" low-value assets, mirroring the process by which words become obsolete in natural language.

- **Market-Driven Architecture:** In this paradigm, the high-level software architecture is not dictated by a top-down, centralized plan. Instead, it emerges from the aggregate of local, economically-driven decisions made by thousands of autonomous agents. Architectural patterns are not enforced by human review but are discovered and propagated by the market. If a particular way of composing functions (e.g., a specific set of Rule FSTs for handling web requests) consistently produces high-utility, low-risk outcomes, agents will rationally favor it. This pattern will outcompete alternatives, spreading through the ecosystem and becoming a de facto standard. The system's architecture is thus a constantly evolving reflection of what the "market" has determined to be most efficient and robust.

The platform on which these agents operate is not a neutral environment; it is an active **"choice architect"**.[55] Drawing from the insights of behavioral economics, the system's designers can "nudge" the collective behavior of the agent population toward globally desirable outcomes without resorting to rigid, top-down commands that would stifle their autonomy.[56] The agent platform can manipulate the "choice environment" in subtle but powerful ways:

- **Framing and Salience:** When an agent queries the function space for a dependency, the results can be framed to highlight risk. Instead of just showing performance metrics, the UI could make the function's beta score highly salient, or frame the choice in terms of potential losses (exploiting loss aversion): "Library A is 5% faster, but Library B avoids a dependency with a 10% chance of a critical failure".[55]
- **Setting Smart Defaults:** The system can set intelligent defaults. For example, when an agent adds a new dependency, the default version selected could be the most stable, lowest-beta version, rather than simply the latest version, forcing the agent to consciously "opt-in" to higher risk.[55]
- **Leveraging Social Proof:** The system can display information about the choices of other agents: "85% of agents in high-security projects chose this implementation".[56] This leverages the tendency of agents (like humans) to follow established norms, reinforcing best practices organically.

Through these "digital nudges," a human overseer or a meta-level policy agent can guide the macro-level evolution of the software ecosystem toward properties like lower systemic risk, higher security, and better maintainability. This is achieved by manipulating the choice architecture at the micro-level of individual agent decisions, providing a powerful and scalable mechanism for system governance that respects the distributed, autonomous nature of the

agent collective.

Section 8. Future Directions: From Agentic Coding to Self-Evolving Software

The synthesis of linguistic composition and economic optimization creates a paradigm with the potential to fundamentally reshape the future of software engineering. This section extrapolates the long-term implications of this framework, envisioning a future where software systems are not merely built but are cultivated as self-organizing, self-improving, and self-evolving digital economies.

8.1. Autonomous Management of Technical Debt and Systemic Risk

By treating functions as assets and the system as a portfolio, we can create a dynamic "balance sheet" for the software project. Functions with positive long-term utility are assets. Functions with negative long-term utility—the embodiment of technical debt—are liabilities. This quantification enables a new level of autonomous system maintenance.

Specialized "refactoring" agents could be tasked with managing this balance sheet. Their goal would be to autonomously identify and "refinance" the system's debt. They would proactively search for low-utility, high-maintenance assets and replace them with higher-quality alternatives, constantly improving the health of the codebase without direct human intervention. Similarly, the system could monitor its own aggregate systemic risk in real-time by tracking the average beta of its components. If the exposure to a particular high-risk hub dependency becomes too great, the system could autonomously decide to mitigate this risk by forking and maintaining its own version of the library or by commissioning agents to build a lower-beta alternative.

8.2. The Emergence of Coding "Dialects" and Specialization

Just as human communities develop specialized vocabularies and jargon, it is likely that groups of agents working on specific subdomains of a large project would develop their own coding "dialects." A team of agents focused on user interface development might evolve a preferred set of high-level compounded functions and architectural patterns (Rule FSTs)

optimized for rendering and state management. A different team working on data processing might develop a completely different "dialect" built around primitives for parallel computation and data transformation. This specialization would lead to the emergence of highly efficient, domain-specific sub-ecosystems within the larger project, each with its own optimized "language" for solving its particular class of problems.

8.3. A New Paradigm for Human-AI Interaction

Perhaps the most profound implication of the linguo-economic paradigm is the redefinition of the human developer's role. In this future, the primary task of the human is no longer the line-by-line writing of code. Instead, the human transitions to the role of a **system governor, an economic planner, or a central banker** for the agent society.

The human's responsibilities would shift to a higher level of abstraction:

- **Defining Value:** The human sets the high-level objectives by designing the multi-attribute utility function. They decide the relative weights of performance, security, maintainability, and other quality attributes, thereby defining what "value" means for the system.
- **Setting Monetary Policy:** The human controls the "money supply" by allocating the system's scarce resources (compute, LLM tokens). By making resources more or less expensive, they can encourage or discourage "spending," controlling the overall pace of development.
- **Managing Risk:** The human sets the global risk tolerance for the system, defining the acceptable level of systemic risk (beta) for the project's dependency portfolio.
- **Designing the Market:** Most importantly, the human acts as a **choice architect** and **mechanism designer**. They design the rules of the game—the auction mechanisms, the reputation systems, the negotiation protocols, and the digital nudges—that shape the interactions of the agent collective and guide its emergent behavior toward the desired goals.

The following table provides a comparative analysis to position the Linguo-Economic Agentic (LEA) paradigm within the broader context of software engineering history, highlighting its unique characteristics and potential advantages.

Table 3: A Comparative Analysis of Coding Paradigms

Dimension	Procedural	Object-Oriented (OOP)	Functional (FP)	Linguo-Economic Agentic (LEA)

Primary Unit	Procedure/Function	Object (Data + Methods)	Pure Function	**Compounded Function (Asset)**
Modularity	Low; based on files.	High; based on classes.	High; based on functions.	**Very High; based on verifiable composition of primitives.**
State Management	Global state.	Encapsulated state.	Immutable state.	**State managed by agents; changes are economic transactions.**
Risk Management	Ad-hoc; testing.	Ad-hoc; testing, design patterns.	Type systems, purity.	**Explicit & Quantitative; modeled via CAPM (Beta), managed as a portfolio.**
Concurrency	Difficult; locks, mutexes.	Difficult; locks, actors.	Easier; no shared state.	**Managed by agents via negotiation and resource allocation games.**
Evolution	Manual refactoring.	Manual refactoring.	Manual refactoring.	**Autonomous; driven by market forces of utility and risk.**
Human Role	Coder	Designer/Architect	Composer of Functions	**Economist / Choice Architect**

Conclusions

This report has introduced a novel theoretical framework for AI agentic coding, grounded in the dual neoclassical principles of linguistic compounding and economic theory. By synthesizing these two powerful paradigms, we have outlined a path toward a new generation of autonomous software development systems that are not only generative but also rational, adaptive, and self-optimizing.

The linguistic framework, which models function creation as a form of neoclassical compounding formalized by Finite-State Transducers, offers a route to generating code that is modular, architecturally compliant, and formally verifiable by construction. The economic framework, which treats functions as assets and agents as rational actors, provides a robust set of tools for optimizing resource allocation, managing systemic risk through a portfolio-based approach, and designing incentive structures that foster effective collaboration.

The fusion of these frameworks in the proposed generative-evaluative loop gives rise to an emergent software ecosystem that behaves like a living language and a dynamic market. In this ecosystem, the architecture evolves, technical debt is autonomously managed, and the very role of the human developer is elevated from a crafter of code to an architect of a digital economy. While the practical implementation of this vision presents significant research and engineering challenges, the linguo-economic paradigm offers a coherent and compelling theoretical foundation for the future of self-evolving software systems. It suggests that the key to unlocking the next level of autonomous software development lies not just in more powerful generative models, but in endowing those models with the structural grammar of language and the rational principles of economics.

Works cited

1. Neoclassical compounds in the onomasiological approach (Chapter ..., accessed on October 2, 2025, https://www.cambridge.org/core/books/semantics-of-compounding/neoclassical-compounds-in-the-onomasiological-approach/1219455B4A9A4F02AE7A7B19C827B6D5
2. Neoclassical Compounding in the Romance Languages | Oxford Research Encyclopedia of Linguistics, accessed on October 2, 2025, https://oxfordre.com/linguistics/display/10.1093/acrefore/9780199384655.001.0001/acrefore-9780199384655-e-456?d=%2F10.1093%2Facrefore%2F9780199384655.001.0001%2Facrefore-9780199384655-e-456&p=emailAUYqv9Ko4NoeY
3. Anzeige von Neoclassical compounds and final combining forms in English |

Linguistik Online, accessed on October 2, 2025, https://bop.unibe.ch/linguistik-online/article/view/1631/2754
4. MOR_022 - Linguistic Micro-Lectures: Neoclassical Compounds - YouTube, accessed on October 2, 2025, https://www.youtube.com/watch?v=9f2pa8cn3fc
5. ON THE PARALLEL BETWEEN NEOCLASSICAL COMPOUNDS IN ENGLISH AND MODERN GREEK, accessed on October 2, 2025, https://pasithee.library.upatras.gr/pwpl/article/download/15/12
6. Neoclassical compound - Wikipedia, accessed on October 2, 2025, https://en.wikipedia.org/wiki/Neoclassical_compound
7. Neoclassical compounds and final combining forms in English - ResearchGate, accessed on October 2, 2025, https://www.researchgate.net/publication/307688738_Neoclassical_compounds_and_final_combining_forms_in_English
8. Composite pattern - Wikipedia, accessed on October 2, 2025, https://en.wikipedia.org/wiki/Composite_pattern
9. Composite - Refactoring.Guru, accessed on October 2, 2025, https://refactoring.guru/design-patterns/composite
10. 600.405 / 600.406: Finite-State Methods in NLP, accessed on October 2, 2025, https://www.cs.jhu.edu/~jason/405/
11. Ling/CSE 472: Introduction to Computational Linguistics - faculty.washington.edu, accessed on October 2, 2025, https://faculty.washington.edu/ebender/2020_472/0409.pdf
12. Lecture 4: Finite State Transducers for Morphological Analysis, accessed on October 2, 2025, https://www.krchowdhary.com/nlsp-lect/nlplect4.pdf
13. 1.5 Finite State Methods in Computational Linguistics and their Limitations - CS @ Union, accessed on October 2, 2025, https://cs.union.edu/~striegnk/courses/nlp-with-prolog/html/node6.html
14. Finite State Languages - Linguistics - Oxford Bibliographies, accessed on October 2, 2025, https://www.oxfordbibliographies.com/abstract/document/obo-9780199772810/obo-9780199772810-0181.xml
15. MorphologicalAnalysisTutorial - foma - A self-contained tutorial for ..., accessed on October 2, 2025, https://fomafst.github.io/morphtut.html
16. Finite-state methods and models in natural language processing, accessed on October 2, 2025, https://www.cambridge.org/core/journals/natural-language-engineering/article/finitestate-methods-and-models-in-natural-language-processing/E05981ED7BD93C062DBCE7218C236728
17. Finite State Transducer (FSTs) in NLP - GeeksforGeeks, accessed on October 2, 2025, https://www.geeksforgeeks.org/nlp/finite-state-transducer-fsts-in-nlp/
18. 2.3 Morphological Analysis with Finite State Transducers - CS @ Union, accessed on October 2, 2025, https://cs.union.edu/~striegnk/courses/nlp-with-prolog/html/node14.html
19. Local Success Does Not Compose: Benchmarking Large Language Models for Compositional Formal Verification - arXiv, accessed on October 2, 2025,

https://arxiv.org/html/2509.23061v1
20. Re:Form — Reducing Human Priors in Scalable Formal Software Verification with RL in LLMs: A Preliminary Study on Dafny - arXiv, accessed on October 2, 2025, https://arxiv.org/html/2507.16331v2
21. agree - Loonwerks, accessed on October 2, 2025, http://loonwerks.com/tools/agree.html
22. Compositional Reasoning Gotchas in Practice - Texas Computer Science, accessed on October 2, 2025, https://www.cs.utexas.edu/~hunt/FMCAD/FMCAD15/papers/paper59.pdf
23. Formal Verification of McMillan's Compositional Assume-Guarantee Rule - Computer Science Laboratory - SRI International, accessed on October 2, 2025, https://www.csl.sri.com/~rushby/papers/mcmillan.pdf
24. Understanding Cross-Lingual Alignment—A Survey - arXiv, accessed on October 2, 2025, https://arxiv.org/html/2404.06228v2
25. Understanding Cross-Lingual Alignment—A Survey - ACL Anthology, accessed on October 2, 2025, https://aclanthology.org/2024.findings-acl.649.pdf
26. What is the difference between multilingual and cross-lingual in the context of NLP? - Reddit, accessed on October 2, 2025, https://www.reddit.com/r/LanguageTechnology/comments/f3epzu/what_is_the_difference_between_multilingual_and/
27. Cross-Lingual Semantic Alignment in Large Language Models via Context-Aware Training, accessed on October 2, 2025, https://www.preprints.org/manuscript/202503.0935/v1
28. Cross-Lingual Semantic Alignment in Large Language Models via Context- Aware Training - Preprints.org, accessed on October 2, 2025, https://www.preprints.org/frontend/manuscript/5c39cb5348a0f25365217ecabc19f56e/download_pub
29. Align after Pre-train: Improving Multilingual Generative Models with Cross-lingual Alignment | OpenReview, accessed on October 2, 2025, https://openreview.net/forum?id=3PaVCdeEmW
30. Marginal utility | Research Starters - EBSCO, accessed on October 2, 2025, https://www.ebsco.com/research-starters/economics/marginal-utility
31. Understanding Marginal Utility: Definition, Types, and Economic Impact - Investopedia, accessed on October 2, 2025, https://www.investopedia.com/terms/m/marginalutility.asp
32. Marginal utility - Wikipedia, accessed on October 2, 2025, https://en.wikipedia.org/wiki/Marginal_utility
33. What Is the Law of Diminishing Marginal Utility? - Investopedia, accessed on October 2, 2025, https://www.investopedia.com/terms/l/lawofdiminishingutility.asp
34. Software Dependency Graphs: Definition, Use Cases, and Implementation - PuppyGraph, accessed on October 2, 2025, https://www.puppygraph.com/blog/software-dependency-graph
35. How to Use a Dependency Graph to Analyze Dependencies - Jit.io, accessed on October 2, 2025,

https://www.jit.io/resources/app-security/how-to-use-a-dependency-graph-to-analyze-dependencies
36. arxiv.org, accessed on October 2, 2025, https://arxiv.org/html/2508.13819v1
37. (PDF) Structural and Connectivity Patterns in the Maven Central Software Dependency Network - ResearchGate, accessed on October 2, 2025, https://www.researchgate.net/publication/394687969_Structural_and_Connectivity_Patterns_in_the_Maven_Central_Software_Dependency_Network
38. Propagation of Bankruptcy Risk over Scale-Free Economic Networks - MDPI, accessed on October 2, 2025, https://www.mdpi.com/1099-4300/24/12/1713
39. No Free Lunch For Programmers: Digital Supply Chains and the Economics of Software Dependency Management - Sam Boysel, accessed on October 2, 2025, https://sboysel.github.io/pdf/oss_digital_supply_chains_sboysel_article.pdf
40. Machine learning-driven root cause analysis and predictive defect prevention in enterprise insurance software - World Journal of Advanced Research and Reviews, accessed on October 2, 2025, https://wjarr.com/sites/default/files/WJARR-2024-0485.pdf
41. Some Vital Lessons in How Systemic Risk is Changing Cybersecurity | MUFG Americas, accessed on October 2, 2025, https://www.mufgamericas.com/insights-and-experience/trending-topics/some-vital-lessons-in-how-systemic-risk-changing-in-cybersecurity
42. Systemic Cyber Risk: A Primer | Carnegie Endowment for International Peace, accessed on October 2, 2025, https://carnegieendowment.org/research/2022/03/systemic-cyber-risk-a-primer?lang=en
43. accessed on January 1, 1970, https://www.carnegieendowment.org/research/2022/03/systemic-cyber-risk-a-primer?lang=en
44. Cascading failure - Wikipedia, accessed on October 2, 2025, https://en.wikipedia.org/wiki/Cascading_failure
45. Capital asset pricing model (CAPM) | Research Starters - EBSCO, accessed on October 2, 2025, https://www.ebsco.com/research-starters/business-and-management/capital-asset-pricing-model-capm
46. Capital asset pricing model - Wikipedia, accessed on October 2, 2025, https://en.wikipedia.org/wiki/Capital_asset_pricing_model
47. What is the role of game theory in multi-agent systems? - Milvus, accessed on October 2, 2025, https://milvus.io/ai-quick-reference/what-is-the-role-of-game-theory-in-multiagent-systems
48. How Game Theory Shapes Modern Multi-Agent AI Systems | by Tiyasa Mukherjee | Medium, accessed on October 2, 2025, https://medium.com/@mukherjeetiyasa1998/game-theoretic-impact-on-multi-agent-systems-4307c3e8872f
49. milvus.io, accessed on October 2, 2025, https://milvus.io/ai-quick-reference/what-is-the-role-of-game-theory-in-multiag

ent-systems#:~:text=A%20key%20application%20of%20game,might%20compete%20for%20computational%20resources.
50. Applications in artificial intelligence and multi-agent systems | Game ..., accessed on October 2, 2025, https://fiveable.me/game-theory/unit-14/applications-artificial-intelligence-multi-agent-systems/study-guide/DVaH3Br8LSGzIAW4
51. Bounded rationality in Keynesian beauty contests: a lesson for central bankers?, accessed on October 2, 2025, https://www.researchgate.net/publication/337089669_Bounded_rationality_in_Keynesian_beauty_contests_a_lesson_for_central_bankers
52. The Agent Paradigm: From Theory to Implementation in Modern AI, accessed on October 2, 2025, https://medium.com/@ysakuragi16/the-agent-paradigm-from-theory-to-implementation-in-modern-ai-2-0690dfe01b2d
53. Painful intelligence: What AI can tell us about human suffering - arXiv, accessed on October 2, 2025, https://arxiv.org/html/2205.15409v2
54. A Agent Based Computational Economics, accessed on October 2, 2025, https://www.complexity.kr/wp-content/uploads/2013/07/ECSS-ABMScompilation.pdf
55. (PDF) Digital Nudging: Guiding Online User Choices through ..., accessed on October 2, 2025, https://www.researchgate.net/publication/320419336_Digital_Nudging_Guiding_Online_User_Choices_through_Interface_Design
56. What are Behavioral Economics — updated 2025 | IxDF - The Interaction Design Foundation, accessed on October 2, 2025, https://www.interaction-design.org/literature/topics/behavioral-economics
57. (PDF) 23 Ways to Nudge: A Review of Technology-Mediated Nudging in Human-Computer Interaction - ResearchGate, accessed on October 2, 2025, https://www.researchgate.net/publication/332745321_23_Ways_to_Nudge_A_Review_of_Technology-Mediated_Nudging_in_Human-Computer_Interaction